

UdaanHer Saathi MVP: Technical Specification

Version 1.0 — the contract every task in the Task Book is built against

Team: Aishi + mentor • 9 July 2026

Contents

1 Purpose and scope	2
2 System overview	2
3 Technology stack (pinned)	2
4 Repository layout	3
5 Configuration and secrets	4
6 External API contracts (verified 9 July 2026)	5
6.1 Sarvam speech-to-text (Saaras v3)	5
6.2 Sarvam text-to-speech (Bulbul v3)	5
6.3 OpenAI	5
7 Internal API contract (backend ↔ frontend)	5
7.1 GET /health	5
7.2 POST /api/session	5
7.3 POST /api/turn	6
7.4 GET /api/learner/{learner_id}/progress	6
8 UI command protocol (the agent drives the screen)	6
9 Agent specification (LangGraph)	7
9.1 State machine	7
9.2 AgentState (the object flowing through the graph)	7
9.3 Prompting rules (apply to every node prompt)	8
10 Data model (SQLite)	8
11 Content schemas	8
11.1 Curriculum (content/curriculum/tailoring.json)	9
11.2 Rubrics (content/rubrics/tailoring_rubrics.json)	9
11.3 Visual-aid index (content/visual_aids/index.json)	9
12 Languages	9
13 Consent, dignity, and data safety	9

14 Engineering conventions	10
15 MVP acceptance criteria (the demo checklist)	10

1. Purpose and scope

This document is the single source of truth for *how* the UdaanHer Saathi MVP is built: the stack, the repository layout, every API contract, every data schema, and the agent’s state machine. The companion document, *UdaanHer Saathi MVP: Task Book*, breaks the work into sequenced tasks; each task references sections of this spec by number. If a task and this spec ever disagree, this spec wins; if this spec is wrong, change the spec first, then the code.

What we are building (from the plan document `udaanher_mentor_mvp_plan.pdf`): a voice-first web application in which an AI mentor converses with a low-literacy woman in her own language and drives the entire screen itself. Two features, end to end:

1. **Feature 1 — onboarding:** the mentor gets to know her by voice (name, background, interests, informal skill assessment) and saves a persistent learner profile.
2. **Feature 2 — teaching loop:** the mentor teaches one tailoring lesson step by step, checks understanding through a conversational viva, re-teaches whatever is shaky, and records progress.

Non-goals for the MVP (do not build, even if easy): kiosk hardware, facial login, employer marketplace, mock interviews, a numeric readiness score, offline mode, more than three languages at demo time, user-visible typing anywhere in the learner flow.

2. System overview

Three deployable parts and two external services:

Part	Responsibility
Frontend (browser)	Records microphone audio, plays mentor audio, renders whatever UI command the agent sends. It has <i>no</i> navigation logic of its own.
Backend (FastAPI)	The harness. Exposes the API in §7, calls Sarvam and OpenAI, runs the LangGraph agent, owns the database.
SQLite database	Learner profiles, sessions, turns, per-concept mastery (§10).
Sarvam AI	Ears and mouth: Saaras v3 speech-to-text, Bulbul v3 text-to-speech (§6).
OpenAI	The brain: reasoning, conversation, viva grading, via LangGraph (§9).

One conversational turn, end to end: browser records audio → `POST /api/turn` → Saaras transcribes → LangGraph agent produces {reply text, UI command, state updates} → Bulbul synthesises reply audio → response JSON returns to browser → browser plays audio and renders the UI command. Target: under 5 seconds per turn.

3. Technology stack (pinned)

Do not substitute items in this table without a team decision recorded in the repo’s `docs/decisions.md`.

Layer	Choice	Notes
Language (backend)	Python 3.11+	
Web framework	FastAPI + Uvicorn	Async; multipart uploads.
Agent framework	LangGraph (latest 0.x)	With <code>langchain-openai</code> .
LLM	OpenAI <code>gpt-5.1</code>	Or newest flagship at build time; cheaper <code>-mini</code> tier for transcript cleanup only. Verify names at https://platform.openai.com/docs/models .
Validation	Pydantic v2	All request/response and agent output models.
Database	SQLite via <code>SQLModel</code>	Single file <code>data/app.db</code> ; Postgres later if ever needed.
HTTP client	<code>httpx</code>	For Sarvam calls.
Tests	<code>pytest</code>	Fixtures in <code>backend/tests/fixtures/</code> .
Frontend	Vite + React 18 + TypeScript	Plain CSS; no component library.
Audio capture	<code>MediaRecorder</code> API	<code>audio/webm; codecs=opus</code> .
Deployment	Render (backend) + Vercel (frontend), or Render alone serving built frontend	Decided in Task T25.

4. Repository layout

The GitHub repository is a monorepo. Create exactly this skeleton in Task T01:

```

udaanher-saathi/
  README.md          # what this is + how to run both halves in 5 commands
  .gitignore         # python, node, .env, data/*.db
  .env.example       # every variable from Section 5, with dummy values
  docs/
    decisions.md     # one line per stack/scope decision, dated
    spec.pdf         # this document
    tasks.pdf        # the task book
  backend/
    pyproject.toml   # deps pinned
    app/
      main.py        # FastAPI app, routes only
      config.py       # env loading (pydantic-settings)
      services/
        stt.py        # Sarvam Saaras wrapper
        tts.py        # Sarvam Bulbul wrapper
        llm.py        # OpenAI client factory
      agent/
        graph.py      # LangGraph state machine
        state.py      # AgentState model

```

```

    nodes/                # one file per stage (greet.py, discover.py, ...)
    prompts/              # one .md file per stage prompt
models/
    db.py                 # SQLAlchemy tables (Section 10)
    ui.py                 # UI command models (Section 8)
    api.py                # request/response models (Section 7)
    content/
        loader.py         # loads + validates curriculum JSON
tests/
scripts/
    smoke_sarvam.py        # proves STT+TTS keys work
    smoke_openai.py        # proves LLM key works
    seed_demo.py           # creates the pre-seeded "Sunita" learner
frontend/
    package.json
    src/
        App.tsx            # kiosk shell: talk button + render area
        api.ts             # typed client for Section 7
        types.ts           # mirrors Section 8 UI commands
        components/        # OptionCards.tsx, LessonStep.tsx, VideoEmbed.tsx,
                             # ProfileCard.tsx, ProgressView.tsx

        audio/
            recorder.ts     # MediaRecorder push-to-talk
            player.ts       # base64 -> playback, earcon
content/
    curriculum/tailoring.json    # Section 11.1
    rubrics/tailoring_rubrics.json # Section 11.2
    visual_aids/index.json       # Section 11.3
    assets/                      # diagrams (png/svg), earcon.mp3

```

5. Configuration and secrets

All secrets live in `backend/.env`, which is git-ignored; `.env.example` documents every key. Never commit a real key; never put keys in frontend code (the browser must only ever talk to our backend).

Variable	Meaning
SARVAM_API_KEY	From https://dashboard.sarvam.ai
OPENAI_API_KEY	From https://platform.openai.com
OPENAI_MODEL	e.g. gpt-5.1
TTS_SPEAKER	Bulbul voice id; pick a warm female voice from the dashboard voice list
DEFAULT_LANGUAGE	gu-IN
DATABASE_URL	sqlite:///data/app.db
CORS_ORIGINS	http://localhost:5173 in dev; the deployed frontend URL in prod

6. External API contracts (verified 9 July 2026)

6.1 Sarvam speech-to-text (Saaras v3)

- POST `https://api.sarvam.ai/speech-to-text`
- Header: `api-subscription-key: <SARVAM_API_KEY>`
- Body: `multipart/form-data` with fields `file` (the audio blob; WebM/Opus from the browser is accepted), `model = saaras:v3`, `language_code` (BCP-47, e.g. `gu-IN`; omit for auto-detect), `mode = transcribe`.
- Response: JSON with `transcript` (string), `language_code`, `request_id`.

6.2 Sarvam text-to-speech (Bulbul v3)

- POST `https://api.sarvam.ai/text-to-speech`
- Header: `api-subscription-key: <SARVAM_API_KEY>`
- JSON body: `text` (max 2500 chars), `target_language_code` (one of `bn-IN`, `en-IN`, `gu-IN`, `hi-IN`, `kn-IN`, `ml-IN`, `mr-IN`, `od-IN`, `pa-IN`, `ta-IN`, `te-IN`), `model = bulbul:v3`, `speaker = TTS_SPEAKER` from config, `output_audio_codec = mp3`, `speech_sample_rate = 24000`.
- Response: JSON `{"request_id": ..., "audios": [<base64 mp3>]}`. Decode `audios[0]`.

6.3 OpenAI

Use the official `openai` Python SDK through `langchain-openai`'s `ChatOpenAI` inside `LangGraph` nodes. Every place the agent must produce data (a profile, a grade, a UI command) uses *structured output* bound to a Pydantic model — never free-text parsing. Keep `temperature` low (0–0.4) for grading nodes, moderate (0.7) for conversational nodes.

7. Internal API contract (backend ↔ frontend)

Exactly four endpoints. All responses are JSON. All errors use HTTP status codes plus body `{"error": {"code": string, "message": string}}`; user-facing failure audio is the frontend's job (§??).

7.1 GET /health

Returns `{"status": "ok", "version": string}`. Used by smoke tests and deployment checks.

7.2 POST /api/session

Starts or resumes a session.

```
Request: { "learner_name": string | null,    // null => brand-new visitor
           "pin": string | null,           // spoken 4-digit pin for returning users
           "language": "gu-IN" | "hi-IN" | "en-IN" }
Response: { "session_id": string,
            "learner_id": string | null,    // null until profile is saved
```

```

    "greeting_audio_b64": string,    // mentor speaks first
    "greeting_text": string,
    "ui": UICommand,                // Section 8
    "stage": Stage }                // Section 9

```

7.3 POST /api/turn

One conversational turn. multipart/form-data:

```

Fields:  session_id: string
         audio: file (webm/opus)  OR  tapped_option_id: string
         (exactly one of audio / tapped_option_id must be present:
          a card tap is a turn too)
Response: { "transcript": string | null,    // what she said, null on tap
            "reply_text": string,
            "reply_audio_b64": string,      // mp3
            "ui": UICommand,
            "stage": Stage,
            "latency_ms": { "stt": int, "agent": int, "tts": int } }

```

7.4 GET /api/learner/{learner_id}/progress

Returns the data behind the progress view: lessons completed, per-concept mastery, next lesson. Shape defined by `ProgressPayload` in §8.

8. UI command protocol (the agent drives the screen)

Every agent turn returns exactly one `UICommand`. The frontend is a switch statement over `type`; it never decides on its own what screen comes next. Pydantic models live in `backend/app/models/ui.py`; mirrored TypeScript types in `frontend/src/types.ts`. The two must stay in lockstep — any change is one commit touching both files.

```

UICommand = { "type": "idle" }                // just the talk button
              | { "type": "show_options",
                  "prompt": string,            // large text, also spoken
                  "options": [ { "id": string,
                                "label": string, // in session language
                                "image": string } ] } // path under /content/assets
              | { "type": "show_lesson_step",
                  "lesson_id": string, "step_index": int, "total_steps": int,
                  "image": string, "caption": string } // one line, large font
              | { "type": "show_video",
                  "url": string,                // YouTube embed URL
                  "caption": string }
              | { "type": "show_profile_card",
                  "profile": { "name": string, "village": string,
                              "language": string, "interest": string,
                              "starting_level": "new"|"some"|"experienced",
                              "notes": string } }

```

```
| { "type": "show_progress",
    "payload": ProgressPayload }
```

```
ProgressPayload = { "skill": string,
                    "lessons": [ { "lesson_id": string, "title": string,
                                    "status": "done"|"current"|"locked" } ],
                    "concepts": [ { "concept_id": string, "label": string,
                                    "mastery": "strong"|"shaky"|"unseen" } ],
                    "next_step_text": string }
```

Rendering rules: one command replaces the previous render entirely; text on screen is always *also* spoken (never rely on reading); every option card is answerable by tap *or* voice.

9. Agent specification (LangGraph)

9.1 State machine

The graph’s routing field is **stage**. Allowed values and transitions:

Stage	Responsibility	Moves to
greet	Welcome; ask name; spoken consent to remember her (§13)	discover , or resume if name+PIN matches
discover	Village, current work, interests via show_options	assess
assess	3–4 story-questions about the chosen skill; infer starting_level	confirm_profile
confirm_profile	Read profile back aloud; save on yes; fix on no	teach (or close)
teach	Deliver current lesson step by step; answer interruptions	viva when steps done
viva	Ask the lesson’s rubric questions conversationally; grade each	reteach if any concept shaky, else wrapup
reteach	Re-explain one shaky concept differently (analogy / diagram / video); re-check with one question	back to viva outcome check; max 2 re-teach rounds per concept
wrapup	show_progress ; say what comes next time	close
close	Save everything; goodbye	terminal
resume	Greet returning learner by name with history; jump to her next lesson	teach

Hard rules the graph enforces in code (not in prompts): **teach** is unreachable until a profile row exists; **viva** questions come only from the rubric file; a concept is **shaky** until graded **strong**; after 2 failed re-teach rounds the agent marks the concept “revisit next session” and moves on warmly (never a third drill).

9.2 AgentState (the object flowing through the graph)

```
AgentState = { session_id, learner_id | null, language,
```

```

stage: Stage,
transcript: string,           // this turn's user input
history: last 12 turns,       // rolling window
profile: ProfileDraft | null,
lesson_id | null, step_index: int,
viva: { question_ids_asked: [], grades: {concept_id: grade} },
reteach_counts: {concept_id: int},
reply_text: string,           // outputs, set by nodes
ui: UICommand }

```

Persistence: LangGraph’s SQLite checkpointer keyed by `session_id` carries state across turns; durable facts (profile, mastery, lesson completion) are *additionally* written to the tables in §10 — checkpoints are cache, the database is truth.

9.3 Prompting rules (apply to every node prompt)

Prompts live in `backend/app/agent/prompts/`, one Markdown file per stage, in English (the model replies in the session language — state the target language explicitly in every prompt). Every prompt encodes the persona: a warm, patient elder-sister mentor (“didi”); short sentences; one question at a time; no jargon, no English words when speaking Gujarati/Hindi unless they are common loanwords; never says “test”, “exam”, “score”, “wrong”; on a struggling answer, encourage first, then re-explain. Reply length cap: 2–3 sentences (TTS latency and attention both demand it).

10. Data model (SQLite)

<code>learners</code>	<code>id (uuid pk), name, village, language, pin_hash, interest_skill, starting_level, notes, consent_given_at (datetime), created_at</code>
<code>sessions</code>	<code>id (uuid pk), learner_id (fk, nullable), language, started_at, ended_at</code>
<code>turns</code>	<code>id (pk), session_id (fk), role ('user' 'mentor'), transcript, ui_type, stage, latency_ms_total, created_at</code>
<code>lesson_progress</code>	<code>learner_id (fk), lesson_id, status ('in_progress' 'completed'), completed_at</code>
<code>concept_mastery</code>	<code>learner_id (fk), concept_id, mastery ('strong' 'shaky' 'unseen'), last_checked_at, reteach_count</code>

Deletion: a single function `delete_learner(learner_id)` removes the learner row and cascades to all four other tables. It must exist from the moment `learners` exists (§13).

11. Content schemas

Content is data, not code: a non-programmer must be able to edit these JSON files. `content/loader.py` validates them against Pydantic models at backend startup and refuses to boot on invalid content (fail loud, not in front of Sunita).

11.1 Curriculum (content/curriculum/tailoring.json)

```
{ "skill_id": "tailoring",
  "title": {"en-IN": "Tailoring", "hi-IN": "...", "gu-IN": "..."},
  "lessons": [
    { "lesson_id": "tail-01-measure",
      "title": {...per language...},
      "concepts": [ {"concept_id": "c-body-measure",
                     "label": {...}, "must_land": true}, ...3-5 total ],
      "steps": [ { "step_index": 0,
                    "image": "assets/tail01_step0.png",
                    "teaching_notes": "what the agent should convey,
                                      written for the LLM in English",
                    "caption": {...per language...} }, ... ],
      "common_mistakes": ["...", "..."] }, ... ] }
```

Source material: the NSDC Qualification Pack for sewing machine operator / assistant tailor (<https://nqr.gov.in>). Three lessons for the MVP.

11.2 Rubrics (content/rubrics/tailoring_rubrics.json)

Per lesson, per concept: 2–3 viva questions with, for each, **sounds_right** (2–3 examples of a correct answer as a non-literate learner would *speak* it, informal, code-mixed) and **sounds_confused** (common misconception phrasings). The viva node passes these to the LLM as grading context; grades are **strong** or **shaky**, nothing finer.

11.3 Visual-aid index (content/visual_aids/index.json)

```
[ { "concept_id": "c-body-measure",
    "kind": "video" | "diagram",
    "url_or_path": "https://youtube.com/embed/..." | "assets/....png",
    "language": "hi-IN" | "gu-IN" | "any",
    "note": "why this one is good" }, ... ]
```

Rule: 2+ aids per concept, hand-checked by a human (watched/viewed, correct, respectful). The re-teach node may *only* pick from this index — never a live search.

12. Languages

Ship **gu-IN**, **hi-IN**, **en-IN**. Design for five (**+mr-IN**, **bn-IN**): every user-visible string is either generated by the LLM in-language at runtime or looked up from the per-language maps in content JSON — no hard-coded Gujarati/Hindi strings in code. Adding a language must require only: new keys in content JSON, one entry in a **SUPPORTED_LANGUAGES** list, and a Bulbul voice check.

13. Consent, dignity, and data safety

Non-negotiables, in scope for the MVP: (1) before saving a profile, the mentor asks aloud, in her language, whether she agrees that the app remembers her, and stores **consent_given_at** only on a spoken/tapped yes; on no, the session continues without any **learners** row and nothing persists

past the session. (2) `delete_learner` exists and works from day one. (3) No biometrics anywhere. (4) Keys server-side only. (5) Audio recordings are transient — transcripts are stored, raw audio is discarded after transcription.

14. Engineering conventions

- **Git:** small branches named `task/T##-slug`, PR into `main`, at least one other person reads the diff. `main` must always run. Commit messages: `T## short description`.
- **Tests:** every service module and every agent node gets a `pytest`; LLM-dependent tests mock the LLM (assert on prompts and structured-output handling), plus a small number of live smoke tests marked `@pytest.mark.live` that are run manually.
- **Style:** `ruff` for Python lint+format, `prettier` for the frontend; both wired into CI later, run locally from day one.
- **Latency log:** every `/api/turn` response carries its `latency_ms` breakdown, and the backend logs it; we watch this number from the first walking skeleton onward.

15. MVP acceptance criteria (the demo checklist)

The MVP is done when all of the following pass on the deployed URL, on a laptop and on one Android phone, on ordinary Wi-Fi:

1. A new visitor completes onboarding in Gujarati entirely by voice+taps: greeted, consented, discovered, assessed, profile read back and saved. No typing, no menus.
2. Restarting the browser and returning with name+PIN resumes: greeting by name, correct history.
3. The pre-seeded learner completes lesson `tail-01`: all steps taught with visuals, viva held, at least one deliberately-wrong answer triggers a visible re-teach (different explanation or video), progress view reflects reality afterwards.
4. Median turn latency $\leq 5s$ over 20 consecutive turns; a thinking earcon plays while waiting.
5. The same flows work with language switched to Hindi and English.
6. `delete_learner` run against a test learner removes every row.
7. The 5-minute demo script from the plan document has been rehearsed twice end to end, with backup recordings ready.